

# 1 / 3 · CIFAR 전처리와 ResNet 학습

이미지와 정답을 모델에 맞게 만들고, 분류 손실·정확도에 연결한다.

실행: PIL → 증강 → Tensor [C,H,W] → batch → logits [B,K] | 역추적: 정확도 ← 예측 ID + label

## [기출 복기] 학습 transform 전체를 복원

```
train_tfms = T.Compose([
    T.RandomCrop(32, padding=4),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize(CIFAR10_MEAN, CIFAR10_STD),
])
test_tfms = T.Compose([
    T.ToTensor(),
    T.Normalize(CIFAR10_MEAN, CIFAR10_STD),
])
train_set = torchvision.datasets.CIFAR10(
    root=data_root, train=True, download=True, transform=train_tfms)
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
```

**인자 출처** 32: CIFAR 이미지 크기, padding=4: 주어진 증강 설정. CIFAR10\_MEAN/STD: 셀 앞 RGB 평균·표준편차. data\_root·batch\_size: 앞선 설정. transform에는 생성한 변환 객체를 연결.

**실수 방지** 8-bit PIL → ToTensor는 float [3,32,32], 0~1. Normalize는 채널별 (x-mean)/std이며 출력 범위가 정확히 -2.5~2.5로 고정되지는 않습니다. 테스트에는 임의 crop/flip을 넣지 않습니다.

근거: 01\_ResNet18\_CIFAR10.ipynb · 셀 6

## [범위 핵심] CIFAR용 head·손실·정확도

```
model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
                        bias=False)
model.maxpool = nn.Identity()
model.fc = nn.Linear(model.fc.in_features, num_classes)
criterion = nn.CrossEntropyLoss(label_smoothing=cfg.label_smoothing)
logits = model(images)
loss = criterion(logits, labels)
preds = logits.argmax(dim=1)
acc = (preds == labels).float().mean().item()
```

**인자 출처** num\_classes=len(train\_set.classes), CIFAR는 10. fc.in\_features: 기존 모델 head가 받는 특징 수. images,labels: loader의 현재 배치. cfg: TrainConfig 인스턴스.

**실수 방지** logits [B,K] float / labels [B] long. CE 전에 softmax 불필요. ResNet 잔차는 F(x)+shortcut(x), 덧셈 shape가 같아야 합니다. 일반 학습은 zero\_grad→backward→step; 원본 AMP는 scaler.scale(loss).backward()→scaler.step(optimizer)→scaler.update().

근거: 01\_ResNet18\_CIFAR10.ipynb · 셀 9,11,13

가리고 쓰기: Compose 4개 순서 / Normalize 인자 출처 / fc 입력 차원 / argmax 축 / label dtype.

## 2 / 3 · ViT와 Attention·Encoder

이미지를 패치 토큰으로 바꾸고 head별 Attention 뒤 잔차 연결을 구현한다.

실행: 이미지 → patch → CLS+position → Encoder → class | 역추적: class ← CLS ← attention tokens

### [범위 핵심] Patch/CLS/Position

```
num_patches = (image_height // patch_height) * (image_width // patch_width)
patch_dim = cfg.channels * patch_height * patch_width
# 펼친 patch 뒤 nn.Linear(patch_dim, cfg.dim)
x = self.to_patch_embedding(img)
b, n, _ = x.shape
cls_tokens = repeat(self.cls_token, "1 1 d -> b 1 d", b=b)
x = torch.cat((cls_tokens, x), dim=1)
x = x + self.pos_embedding[:, :(n + 1)]
x = self.dropout(x)
x = self.transformer(x)
x = x[:, 0] if self.pool == "cls" else x.mean(dim=1)
return self.mlp_head(x)
```

**인자 출처** image/patch 크기·channels·dim: ViTConfig. b,n: patch embedding 결과 [B,N,D]. cls\_token: \_\_init\_\_의 [1,1,D] 파라미터. pos\_embedding: [1,N+1,D].

**실수 방지** CLS는 토큰 축 dim=1에 추가, 위치 개수 N+1. pool="cls"는 0번 토큰 선택 [B,D]. 32×32, patch=4×4, C=3이면 N=64, patch\_dim=48 (계산 예시).

근거: 02\_ViT\_CIFAR10.ipynb · 셀 11 ViT

### [기출 복기 관련 핵심] QKV 분리와 Encoder 잔차

```
# Attention.forward: h=self.heads, scale=dim_head**-0.5
qkv = self.to_qkv(x).chunk(3, dim=-1)
q, k, v = map(lambda t: rearrange(t,
    "b n (h d) -> b h n d", h=h), qkv)
dots = einsum("b h i d, b h j d -> b h i j", q, k) * self.scale
attn = self.attend(dots)
out = einsum("b h i j, b h j d -> b h i d", attn, v)
out = rearrange(out, "b h n d -> b n (h d)")
return self.to_out(out)
# Transformer.forward (attn/ff는 PreNorm으로 감싼 부품)
for attn, ff in self.layers:
    x = attn(x) + x
    x = ff(x) + x
```

**인자 출처** x: [B,N,D]. self.to\_qkv: Linear(dim,3\*heads\*dim\_head). h=self.heads는 생성자 설정. self.attend=Softmax(dim=-1). attn,ff: self.layers 안 부품.

**실수 방지** Vision Encoder는 보통 전체 패치를 보므로 GPT의 미래 causal mask를 자동으로 넣지 않습니다. 사진 복기의 DETR EncoderLayer는 현재 03\_DETR 노트북에 직접 정의돼 있지 않아, 실제 확인한 ViT Encoder로 공통 구조를 대비합니다. norm 위치는 제시 코드 기준.

근거: 02\_ViT\_CIFAR10.ipynb · 셀 9 Attention / Transformer; DETR MHA·Encoder는 사용자 기출 사진 근거

가리고 쓰기: CLS concat 축 / N+1 이유 / QKV chunk 축 / head 분리·결합 / residual 두 줄.

## 3 / 3 · U-Net·DETR·평가 구분

분류·분할·검출의 출력과 필요한 평가식을 구분한다.

실행: image → spatial features → mask 또는 boxes | 역추적: mask ← decoder + encoder skip

### [범위 핵심] U-Net Up.forward의 skip

```
# forward(self, x1, x2): x1=decoder, x2=encoder skip
x1 = self.up(x1)
diff_y = x2.size(2) - x1.size(2)
diff_x = x2.size(3) - x1.size(3)
x1 = F.pad(x1, [diff_x // 2, diff_x - diff_x // 2,
               diff_y // 2, diff_y - diff_y // 2])
x = torch.cat([x2, x1], dim=1)
return self.conv(x)
# binary mask: logits와 mask 모두 [B,1,H,W]
criterion = nn.BCEWithLogitsLoss()
```

**인자 출처** x1:x2: Up.forward의 두 입력. up-conv: Up.\_\_init\_\_. diff\_x/y: 두 Tensor 공간 크기 차이. BCE 정답 mask는 float 0/1, logits와 같은 shape.

**실수 방지** U-Net은 채널 concat이므로 dim=1, 채널 수는 합계. ResNet의 add와 다릅니다. 이진 mask는 sigmoid(logits)>threshold로 판정. IoU=교집합/합집합, Dice=2×교집합/(예측 수+정답 수). 0분모는 eps로 처리.

근거: 04\_Unet.ipynb · 셀 7 Up, 셀 9 loss; IoU/Dice 식은 공식 평가 범위 보강

### [범위 핵심] DETR class-box 후처리

```
img = transform(im).unsqueeze(0)
outputs = model(img)
probas = outputs["pred_logits"].softmax(-1)[0, :, :-1]
keep = probas.max(-1).values > 0.9
bboxes_scaled = rescale_bboxes(outputs["pred_boxes"][0, keep], im.size)
# box_cxxywh_to_xyxy(x), x:[N,4]
x_c, y_c, w, h = x.unbind(1)
b = [x_c - 0.5*w, y_c - 0.5*h,
     x_c + 0.5*w, y_c + 0.5*h]
return torch.stack(b, dim=1)
```

**인자 출처** im: 원본 PIL 이미지, im.size=(W,H). outputs: 모델 dict. pred\_logits [B,Q,K+1], pred\_boxes [B,Q,4]. 0.9: 코드에 주어진 confidence 기준. rescale은 xyxy에 [W,H,W,H]를 곱함.

**실수 방지** :-1은 no-object 클래스 제외. 같은 keep mask를 boxes에도 적용. DDPM·Stable Diffusion은 이번 마지막 복습에서 후순위: noise-timestep을 모델에 주고 noise 예측 손실로 연결하는 정도를 먼저 기억합니다.

근거: 03\_DETR.ipynb · 셀 8,18

가리고 쓰기: Up의 dim=1 / BCE 입력·정답 shape / IoU와 Dice 분모 / DETR no-object 제외 위치.